

Creative Automation Hub - Project Status

■ COMPLETED

Backend (Go)

■ Go module setup with dependencies ■ Server with Gin framework (port 8080) ■ PostgreSQL connection ■ Redis job queue integration ■ REST API endpoints: **POST** **/api/generate/text** - Text generation **POST** **/api/generate/image** - Image generation **GET** **/api/jobs/:id** - Job status **GET** **/api/assets** - List assets **POST** **/api/brand-kit** - Save brand kit ■ WebSocket server (**/ws**) for real-time updates ■ Job service (queue management) ■ Asset service (content storage) ■ CORS middleware ■ Graceful shutdown AI Workers (Python)

■ Redis queue consumer ■ Text generator (Groq/Anthropic) ■ Image generator (Stability AI/placeholder) ■ Job processing with status updates ■ Multi-variant generation ■ Error handling
Frontend (Next.js 15)

■ TypeScript + Tailwind CSS setup ■ Home page with tab navigation ■ Text generator component ■ Image generator component ■ WebSocket hook for real-time updates ■ Responsive design ■ Loading states & error handling
Documentation

■ README.md ■ ARCHITECTURE.md ■ MVP-DESIGN.md ■ SETUP.md ■ .env.example files ■ .gitignore Scripts

■ PowerShell startup script (start.ps1) ■ TODO
(For Production)

Backend

[] Database schema migrations [] JWT authentication [] Rate limiting [] S3 integration for assets [] Error logging (structured logs) [] Health check improvements [] Unit tests AI Workers

[] Actual Stable Diffusion integration [] Worker pool management [] Retry logic for failed jobs [] Job timeout handling [] Metrics collection Frontend

[] User authentication UI [] Project management [] Brand kit editor [] Asset library with search [] Export functionality [] Canvas editor (Fabric.js) [] Dark mode DevOps

[] Docker Compose for local dev [] Kubernetes manifests [] CI/CD pipeline [] Monitoring (Prometheus/Grafana) ■ Project Structure

```
creative-automation-hub/
├── backend-go/                # Go API (port 8080)
│   ├── cmd/server/           # Main entry
│   │   └── internal/
│   │       ├── handlers/     # HTTP handlers
│   │       ├── services/     # Business logic
│   │       └── models/       # Data models
│   ├── go.mod
│   └── .env.example
├── ai-workers/               # Python workers
│   ├── worker.py             # Main worker
│   ├── text_generator.py     # Text gen (Grog/Claude)
│   ├── image_generator.py    # Image gen (SD/placeholder)
│   ├── requirements.txt
│   └── .env.example
├── frontend/                 # Next.js (port 3000)
│   ├── app/                  # App router
│   ├── components/           # React components
│   ├── hooks/                # Custom hooks
│   ├── package.json
│   └── .env.example
├── README.md
├── SETUP.md
├── start.ps1                 # Startup script
└── .gitignore
```

■ Quick Start

Setup databases:

```
```bash
Start Redis
redis-server # Create PostgreSQL database
```

```
psql -U postgres
CREATE DATABASE creative_hub;
...
```

#### **Configure environment:**

```
```bash
# Backend
cd backend-go
cp .env.example .env # Workers
cd ai-workers
cp .env.example .env
# Add GROQ_API_KEY

# Frontend
cd frontend
cp .env.example .env.local
```
```

#### **Start services:**

```
```bash
# Option 1: Use script
.\start.ps1 # Option 2: Manual
cd backend-go && go run cmd/server/main.go
cd ai-workers && python worker.py
cd frontend && npm run dev
```
```

## **Access:**

- **Frontend:** <http://localhost:3000>
- **Backend:** <http://localhost:8080>
- **Health:** <http://localhost:8080/health> ■ **Key Features**

#### **Text Generation:**

- Multiple variants per request
- Content type: blog, social, ad
- Tone customization
- Real-time progress via WebSocket

#### **Image Generation:**

- Text-to-image
- Style presets
- Batch generation
- Placeholder support (MVP)

#### **Architecture:**

- Go backend for 10x performance
- Python for AI/ML workloads
- Next.js for modern React UI
- Redis for job queue

- PostgreSQL for metadata
- WebSocket for real-time updates

## ■ Performance Targets

**API response: < 50ms WebSocket latency: < 100ms**  
**Text generation: 2-5 seconds Image generation:**  
**5-15 seconds (with Stability AI) Concurrent users:**  
**1000+ ■ Environment Variables**

### **Backend (.env):**

- DB\_HOST, DB\_PORT, DB\_USER, DB\_PASSWORD, DB\_NAME
- REDIS\_HOST, REDIS\_PORT, REDIS\_PASSWORD
- PORT (default: 8080)

### **Workers (.env):**

- REDIS\_HOST, REDIS\_PORT
- LLM\_PROVIDER (groq or anthropic)
- GROQ\_API\_KEY or ANTHROPIC\_API\_KEY
- STABILITY\_API\_KEY (optional)

### **Frontend (.env.local):**

- NEXT\_PUBLIC\_API\_URL (default: http://localhost:8080)
- NEXT\_PUBLIC\_WS\_URL (default: ws://localhost:8080)

## ■ Known Issues

**Database schema not created automatically - run SQL manually Image generation uses placeholder (no Stability AI key) No authentication yet Asset storage in memory (not persisted to S3) ■ Notes**